
```
>>> import random
>>> def returnsTwoTypes():
...     if random.randint(1, 2) == 1:
...         return 42
...     else:
...         return 'forty two'
```

When you're writing code that calls this function, it can be easy to forget that you must handle several possible data types. To continue this example, say we call `returnsTwoTypes()` and want to convert the *number* that it returns to hexadecimal:

```
>>> hexNum = hex(returnsTwoTypes())
>>> hexNum
'0x2a'
```

Python's built-in `hex()` function returns a string of a hexadecimal number of the integer value it was passed. This code works fine as long as `returnsTwoTypes()` returns an integer, giving us the impression that this code is bug free. But when `returnsTwoTypes()` returns a string, it raises an exception:

```
>>> hexNum = hex(returnsTwoTypes())
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
TypeError: 'str' object cannot be interpreted as an integer
```

Of course, we should always remember to handle every possible data type that the return value could have. But in the real world, it's easy to forget this. To prevent these bugs, we should always attempt to make functions return values of a single data type. This isn't a strict requirement, and sometimes there's no way around having your function return values of different data types. But the closer you get to returning only one type, the simpler and less bug prone your functions will be.

There is one case in particular to be aware of: don't return `None` from your function unless your function always returns `None`. The `None` value is the only value in the `NoneType` data type. It's tempting to have a function return `None` to signify that an error occurred (I discuss this practice in the next section,